ASSIGNMENT

LUBABA SADIYA NP

DACS

253132

1.Exploratory data analysis

EDA use many techniques to gain insights about a data

First few raw of data set is:

	rownames	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

2.Feature Engineering

from sklearn.preprocessing import PolynomialFeatures

poly features = PolynomialFeatures(degree=2, include bias=False)

- from EDA ,relationship between x and y are non-linear
- Linear regression assumed that the straight-line relationship is not suitable here
- To get non-linearity ,the original feature x will transformed into:[x,x^2]

3.Model Development

import numpy as np

```
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
# Sample data
x = np.array([[3],[4],[5],[6],[7],[9],[15],[20]])
y = np.array([11,18,27,38,51,83,227,402])
# Train-test split
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
# Polynomial Regression model (degree = 2)
model = Pipeline([
    ('poly_features', PolynomialFeatures(degree=2, include_bias=False)),
    ('linear_model', LinearRegression())
])

# Train model
model.fit(x_train, y_train)
# Predictions
```

y_pred = model.predict(x_test)

This will build a polynomial Regression model using a pipline.

4.Model Validation

Mterics are used here

R^2 score

- It will measure how well the model explains varience in the target variable.
- R^2 is 1(perfect fit)
- R^2 is 0(poor fit)

Code:

Validation code(Train & Test R^2)

train_r2 = model.score(x_train, y_train)

test_r2 = model.score(x_test, y_test)

print("Train R²:", train_r2)

print("Test R²:", test_r2)

code to compute R^2 for multiple degreees

train_scores = []

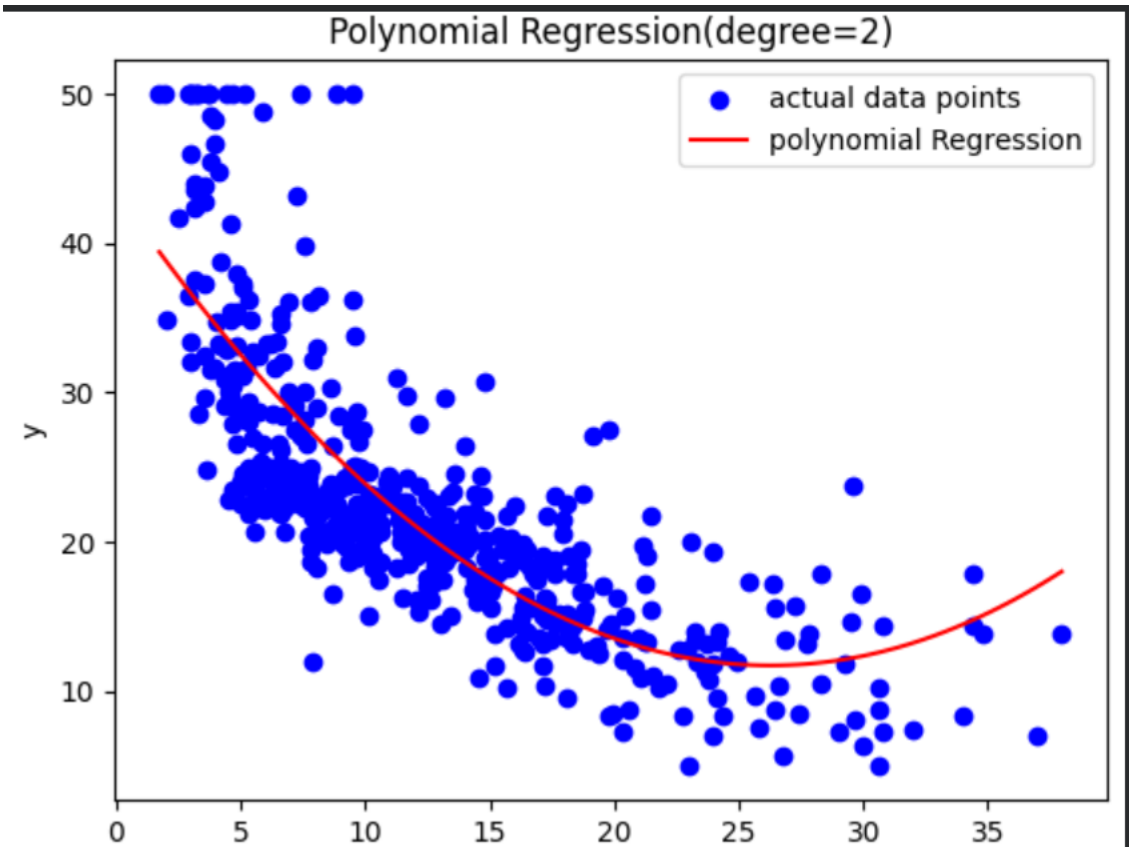
test_scores = []

degrees = range(1, 7)

```
for d in degrees:
    model = Pipeline([
        ('poly_features', PolynomialFeatures(degree=d,
include_bias=False)),
        ('linear_model', LinearRegression())
    ])

    model.fit(x_train, y_train)

    train_scores.append(model.score(x_train, y_train))
    test_scores.append(model.score(x_test, y_test))
```

5.plot

```
plt.plot(degrees, train_scores, marker='o', label='Train R²')

plt.plot(degrees, test_scores, marker='s', label='Test R²')


plt.xlabel('Degree of Polynomial')

plt.ylabel('R² Score')

plt.title('Model Performance vs Polynomial Degree')

plt.legend()

plt.show()
```



6.Insights and Conclusion

- EDA revealed a non-linear relationship it making polynomial regression suitable.
- Polynomial feature engineering improved model performance
- Lower polynomial degrees will underfit the data
- Higher polynomial degree will overfit the data